

Tema 1.

Fundamentos de Python para PLN

Primer prototipo en LinguaAdapt: medir la legibilidad de un texto con Python

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Índice de contenidos

Título narrativo.....	1
Contexto profesional.....	1
Encargo: primer prototipo de legibilidad.....	1
El caso de trabajo.....	1
Qué vas a aprender en esta unidad.....	1
Mapa conceptual de la unidad.....	3
1. Marco teórico.....	3
1.1. Python como herramienta para pensar lingüísticamente con datos.....	3
1.2. <i>Notebooks</i> , Google Colab y narrativas computacionales.....	4
1.3. El texto como dato: de la lectura experta a la cadena de caracteres.....	4
1.4. Variables, objetos y nombres: guardar información lingüística.....	4
1.5. <i>Strings</i> y secuencias: cómo ve Python una cadena de texto.....	5
1.6. Tokenización básica: dividir por espacios no es comprender la lengua.....	5
1.7. Contar como operación lingüística: de la intuición a la medida.....	5
1.8. Puntuación y segmentación: contar oraciones exige una decisión.....	6
1.9. Listas, pertenencia y filtrado.....	6
1.10. Expresiones regulares: buscar patrones formales en el texto.....	6
1.11. Legibilidad: qué puede medir una fórmula y qué no puede medir.....	6
1.12. Reproducibilidad: que otra persona pueda entender y repetir el análisis.....	7
1.13. Del prototipo a la función: por qué reutilizar código importa.....	7
1.14. Interpretación crítica: el número no sustituye el juicio lingüístico.....	7
2. Python para PLN: del texto como dato al primer prototipo de legibilidad.....	8
2.1. Google Colab: el laboratorio de trabajo.....	8
2.2. El texto como dato: variables y <i>strings</i>	8
2.3. De texto continuo a palabras: <i>split()</i>	9
2.4. Contar elementos con <i>len()</i>	9
2.5. Importar herramientas: el módulo <i>string</i>	10
2.6. Recorrer caracteres y detectar puntuación.....	11
2.7. Contar oraciones: una decisión lingüística dentro del código.....	11
2.8. Expresiones regulares: buscar patrones de vocales.....	12

2.9. Construir un índice de legibilidad aproximado.....	13
2.10. Redondear resultados con <code>round()</code>	14
2.11. Reutilizar código: el paso previo a las funciones.....	14
2.12. Cómo mejoraría LinguaAdapt este prototipo.....	15
Errores frecuentes de aprendizaje.....	15
Síntesis final.....	15
Cierre narrativo.....	16
Referencias.....	16

Título narrativo

Primer prototipo en LinguaAdapt: medir la legibilidad de un texto con Python

Contexto profesional

Imagina que te incorporas como estudiante en prácticas a **LinguaAdapt**, una agencia de localización que trabaja con organismos públicos, editoriales y equipos de comunicación institucional. Su especialidad es adaptar textos para que puedan ser comprendidos por públicos diversos: personas con dificultades lectoras, estudiantes que se enfrentan a documentos especializados, ciudadanía que necesita entender información administrativa o equipos que quieren revisar la claridad de sus mensajes.

Encargo: primer prototipo de legibilidad

El equipo no te pide todavía que construyas un sistema avanzado de inteligencia artificial. Tu primer encargo es más concreto y, precisamente por eso, muy formativo: crear un cuaderno en Google Colab que reciba un texto y calcule algunas medidas iniciales de legibilidad.

El caso de trabajo

La petición del cliente es sencilla de formular:

- Cuántas `palabras` tiene el texto.
- Cuántos `signos` de puntuación aparecen.
- Cuántas `oraciones` se pueden estimar.
- Cuántas sílabas contiene aproximadamente.
- Qué índice numérico orientativo puede resumir su dificultad.

Detrás de esa lista hay una idea central para una lingüista de segundo curso: un texto puede leerse como discurso, pero también puede representarse como datos. Python no sustituye la interpretación lingüística. La obliga a hacerse más explícita. Si queremos decir que un texto es «*largo*», «*denso*», «*poco accesible*» o «*formal*», tenemos que decidir qué rasgos observables vamos a medir.

Esta unidad acompaña ese primer cambio de mirada: pasar de leer un texto como lectoras expertas a preparar un pequeño procedimiento computacional que nos ayude a describirlo.

Qué vas a aprender en esta unidad

Al terminar esta guía, deberías poder explicar y aplicar los siguientes conocimientos:

1. Qué es un *notebook* de Google Colab y por qué resulta útil para aprender PLN.
2. Cómo guardar un texto en una variable de Python.
3. Cómo convertir una cadena de texto en una lista de `palabras` con `.split()`.
4. Cómo contar elementos con `len()`.
5. Cómo importar módulos de la biblioteca estándar, como `string` y `re`.
6. Cómo detectar `signos` de puntuación recorriendo carácter por carácter.
7. Cómo estimar `oraciones` a partir de `signos` de puntuación.
8. Cómo usar una expresión regular sencilla para aproximar el número de sílabas.
9. Cómo combinar varias medidas en un índice de legibilidad aproximado.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

10. Cómo interpretar los resultados con criterio lingüístico, sin convertir el número en una verdad absoluta.

Mapa conceptual de la unidad

Pieza del cuaderno	Qué aporta a Python	Qué aporta al PLN
Google Colab	Permite ejecutar código por celdas	Facilita explicar un análisis paso a paso
Variable <code>texto1</code>	Guarda un objeto de tipo <code>string</code>	Representa un texto como dato manipulable
<code>.split()</code>	Divide una cadena en una lista	Hace una tokenización básica por espacios
<code>len()</code>	Cuenta elementos de una colección	Permite medir palabras, signos u oraciones
<code>string.punctuation</code>	Ofrece una lista de signos comunes	Ayuda a localizar puntuación
Listas por comprensión	Construyen listas filtradas	Permiten extraer rasgos del texto
<code>.replace()</code>	Sustituye fragmentos en una cadena	Normaliza signos para contar oraciones
<code>re.findall()</code>	Busca patrones mediante expresiones regulares	Aproxima grupos vocálicos y sílabas
Índice de legibilidad	Combina operaciones numéricas	Resume de forma orientativa la complejidad

1. Marco teórico

Esta unidad introduce el primer gesto intelectual del procesamiento del lenguaje natural: pasar de leer un texto únicamente como discurso a representarlo también como dato manipulable. Para una lingüista, este cambio no significa abandonar la interpretación, sino hacer explícitas algunas decisiones que normalmente se toman de forma intuitiva: qué cuenta como palabra, qué se considera una oración, qué rasgos permiten aproximar la dificultad de lectura y qué límites tiene cualquier medición automática.

El objetivo del marco teórico es preparar la lectura del resto del manual y del cuaderno de trabajo. Antes de ejecutar código, conviene entender por qué Python, los *notebooks*, las cadenas de texto, las listas, las funciones de conteo y las expresiones regulares forman una primera caja de herramientas para observar fenómenos lingüísticos sencillos de forma reproducible.

1.1. Python como herramienta para pensar lingüísticamente con datos

Python es un lenguaje de programación de propósito general, pero en PLN se utiliza porque permite escribir procedimientos legibles, combinar bibliotecas especializadas y trabajar con datos textuales de forma progresiva. Para el alumnado de lingüística, su valor inicial no está en «automatizarlo todo», sino en aprender a formular una pregunta lingüística como una serie de operaciones controlables.

Cuando preguntamos si un texto es largo, claro, denso o accesible, estamos usando categorías interpretativas. Python obliga a traducir parte de esas categorías a operaciones observables: contar palabras, estimar oraciones, localizar signos de puntuación, buscar patrones vocálicos o combinar

varias medidas. Esta traducción nunca es neutra. Cada línea de código presupone una definición operativa del fenómeno que se quiere observar (Biber, Conrad y Reppen, 1998; Gries, 2009).

1.2. Notebooks, Google Colab y narrativas computacionales

Un *notebook* combina código ejecutable, texto explicativo, resultados y documentación en una misma unidad de trabajo. Esta mezcla es especialmente adecuada para el aprendizaje de PLN porque permite justificar una decisión lingüística justo al lado de la celda que la ejecuta. En lugar de separar teoría, procedimiento y resultado en documentos inconexos, el *notebook* convierte el análisis en una narrativa computacional reproducible (Kluyver *et al.*, 2016).

Google Colab es un servicio alojado que permite trabajar con *notebooks* sin instalar un entorno local. Para una primera unidad de programación, esto reduce la barrera técnica: el alumnado puede centrarse en entender variables, cadenas, listas y salidas antes de preocuparse por instalaciones, rutas o versiones. La ventaja didáctica es clara, pero también hay que recordar que un *notebook* depende del orden real de ejecución de sus celdas, no solo del orden visual en el que aparecen.

Elemento	Función didáctica	Riesgo que conviene controlar
Celda de código	Ejecuta instrucciones Python paso a paso	Ejecutar celdas desordenadas y obtener resultados incoherentes
Celda de texto	Documenta decisiones, hipótesis y conclusiones	Escribir explicaciones que no correspondan al código ejecutado
Salida	Muestra resultados intermedios o finales	Confundir un número con una interpretación completa
Historial de ejecución	Permite repetir el análisis	Olvidar que una variable puede proceder de una celda anterior

1.3. El texto como dato: de la lectura experta a la cadena de caracteres

En lingüística, un texto puede estudiarse como discurso, como acto comunicativo, como objeto retórico o como producto social. En Python, para poder manipularlo, primero se representa como una cadena de caracteres. Esa cadena no contiene por sí sola toda la riqueza lingüística del texto: contiene una secuencia ordenada de símbolos. El análisis computacional empieza cuando decidimos qué operaciones aplicar a esa secuencia.

La documentación de Python define el texto como datos de tipo `str`, es decir, cadenas o secuencias textuales. Esto significa que Python puede almacenar un fragmento institucional, recorrer sus caracteres, dividirlo, buscar patrones y calcular longitudes. Para PLN, esta representación es el punto de entrada a operaciones más avanzadas: tokenización, normalización, segmentación, análisis morfológico o extracción de rasgos (Python Software Foundation, 2026b).

1.4. Variables, objetos y nombres: guardar información lingüística

Una variable es un nombre que apunta a un objeto. En la unidad, `texto1` es el nombre que se usa para guardar un texto institucional. La variable no es el texto en sentido lingüístico; es una etiqueta de programación que permite recuperar y manipular un objeto de tipo texto. Esta diferencia es fundamental para evitar errores: Python no interpreta `texto1` como una palabra española, sino como un identificador.

Nombrar bien las variables forma parte del trabajo profesional. En lingüística computacional, nombres como `texto_original`, `texto_adaptado` o `texto_traducido` ayudan a conservar trazabilidad. La trazabilidad permite saber qué versión del texto se está midiendo, qué transformación se ha aplicado y

qué resultado corresponde a cada paso. El código legible no es un lujo estético; es una condición para que otra persona pueda revisar el análisis.

1.5. Strings y secuencias: cómo ve Python una cadena de texto

Un `string` es una secuencia de caracteres. Esto implica que Python puede tratarlo como un conjunto ordenado: puede recorrerlo carácter por carácter, calcular su longitud o aplicar métodos que devuelven nuevas cadenas o listas. En esta unidad se usan comillas triples para almacenar un texto de más de una línea, porque permiten conservar fragmentos extensos sin partarlos en muchas instrucciones.

Para una lingüista, la idea de secuencia es familiar: las lenguas se realizan en cadenas lineales de formas. La diferencia es que Python no sabe qué es una palabra, una sílaba o una oración hasta que le damos una regla operativa. Por eso el trabajo inicial consiste en pasar de la secuencia de caracteres a unidades de análisis más útiles.

1.6. Tokenización básica: dividir por espacios no es comprender la lengua

La tokenización es el proceso de dividir un texto en unidades de análisis llamadas *tokens*. En PLN profesional, un tokenizador puede separar `palabras`, `signos`, abreviaturas, números, contracciones o marcas especiales según reglas lingüísticas y estadísticas. En esta unidad se usa una tokenización inicial muy simple: aplicar el método `.split()`, que separa por espacios en blanco.

Esta estrategia es útil para aprender, pero tiene límites. Si el texto contiene informativa; o ciudadanos., `.split()` conserva la puntuación pegada a la palabra. Por tanto, el resultado no es una segmentación lingüística perfecta, sino una lista razonable para un primer conteo. Esta distinción prepara al alumnado para entender por qué las herramientas de PLN posteriores incorporan tokenizadores más sofisticados (Bird, Klein y Loper, 2009; Jurafsky y Martin, 2026).

Decisión	Regla simple de la unidad	Limitación lingüística
Palabra	Separar por espacios	No separa puntuación pegada ni resuelve abreviaturas
Oración	Dividir por puntos y algunos signos sustituidos	No distingue todos los usos discursivos de la puntuación
Sílaba	Contar grupos vocálicos	No resuelve hiatos, diptongos, sinalefas ni silabificación real
Legibilidad	Combinar promedios de longitud	No mide familiaridad léxica, coherencia ni conocimiento previo

1.7. Contar como operación lingüística: de la intuición a la medida

Contar `palabras`, `signos` u `oraciones` parece una operación elemental, pero en lingüística cuantitativa tiene consecuencias interpretativas. Un conteo convierte una propiedad textual en una variable comparable. Si un texto tiene 25 `palabras` y otro tiene 80, la diferencia de longitud deja de ser solo una impresión. Si un texto tiene muchas `palabras` por oración, podemos formular una hipótesis sobre su complejidad sintáctica.

La función `len()` permite contar elementos de una colección o caracteres de una cadena. Su aparente sencillez es didácticamente muy potente: enseña que una medida depende del objeto que se le entregue. `len(texto1)` cuenta caracteres; `len(palabras)` cuenta elementos de una lista; `len(signos)` cuenta signos filtrados. La misma función responde a preguntas distintas según cómo se haya preparado el dato (Python Software Foundation, 2026a).

1.8. Puntuación y segmentación: contar oraciones exige una decisión

La segmentación de `oraciones` no es una operación puramente mecánica. En un texto escrito, el punto suele marcar frontera oracional, pero también aparece en abreviaturas, cifras, direcciones electrónicas o usos tipográficos. El punto y coma, los dos puntos o los signos de interrogación y exclamación pueden separar unidades discursivas relevantes, pero no siempre equivalen a una oración independiente.

Por eso, cuando el cuaderno reemplaza un punto y coma por un punto antes de dividir el texto, no está haciendo una trampa técnica: está explicitando una decisión lingüística. En este caso, se decide que el punto y coma del ejemplo separa dos segmentos que interesa contar. La regla puede ser razonable para ese texto y demasiado simple para otros. Esta es una idea clave: en PLN, una regla siempre debe evaluarse frente al fenómeno lingüístico concreto.

1.9. Listas, pertenencia y filtrado

Una lista permite almacenar varios elementos en orden. En la unidad se crea una lista de `palabras` y una lista de `signos` de puntuación. Esta forma de representación prepara el terreno para operaciones frecuentes en PLN: filtrar, contar, recorrer y comparar unidades. Cuando se escribe una lista por comprensión, Python crea una nueva lista a partir de una condición.

El operador `in` permite comprobar pertenencia. En el conteo de puntuación, la pregunta es si un carácter del texto pertenece al conjunto de `signos` definido por `string.punctuation`. Esta operación tiene una lectura lingüística sencilla: quedarse solo con los caracteres que se consideran puntuación. La limitación es que `string.punctuation` se basa en un conjunto ASCII de signos; en español y en textos reales pueden aparecer signos tipográficos que exijan una ampliación o normalización específica (Python Software Foundation, 2026c).

1.10. Expresiones regulares: buscar patrones formales en el texto

Las expresiones regulares son patrones formales para localizar secuencias de caracteres. En la unidad se introducen para estimar sílabas mediante grupos vocálicos. El patrón `r'[aeiouáéíóúü]+'` busca uno o más caracteres vocálicos consecutivos, incluyendo vocales acentuadas y u con diéresis. `re.findall()` devuelve todas las coincidencias y permite contarlas (Python Software Foundation, 2026d).

La utilidad lingüística del patrón es clara: muchas sílabas tienen un núcleo vocálico, de modo que contar grupos de vocales puede aproximar el número de sílabas. Pero la aproximación no equivale a silabificación. En español hay diptongos, hiatos, triptongos, sinalefas y fenómenos prosódicos que un patrón tan simple no resuelve. El ejercicio, por tanto, no enseña a silabificar perfectamente; enseña a convertir una hipótesis lingüística en una regla computacional inicial.

1.11. Legibilidad: qué puede medir una fórmula y qué no puede medir

La legibilidad lingüística intenta estimar la facilidad de lectura a partir de rasgos observables del texto. Las fórmulas clásicas suelen combinar longitud media de palabra, número de sílabas y longitud media de oración. La tradición parte de trabajos como el de Flesch (1948) y, para español, de adaptaciones como Fernández Huerta (1959) y Szigriszt Pazos (1993).

La unidad no aplica una fórmula oficial validada, sino un índice aproximado construido con fines didácticos. Esto es importante: una fórmula de legibilidad validada necesita investigación empírica, corpus de referencia y escalas interpretativas. El índice de la unidad sirve para entender cómo se construye una métrica: primero se cuentan rasgos, después se combinan y finalmente se interpreta el resultado con cautela.

Un índice basado en sílabas por palabra y palabras por oración puede orientar, pero no captura toda la dificultad de un texto. No mide conocimiento previo del lector, familiaridad terminológica, cohesión, estructura informativa, ambigüedad, diseño visual ni carga conceptual. Por eso, en comunicación clara y accesibilidad, la medición automática debe acompañarse siempre de lectura experta y revisión con usuarios cuando sea posible.

1.12. Reproducibilidad: que otra persona pueda entender y repetir el análisis

Un análisis reproducible no se limita a obtener un resultado correcto una vez. Debe permitir que otra persona ejecute el mismo procedimiento, revise el orden de las celdas, entienda el significado de las variables y compruebe cómo se obtuvo cada número. En *notebooks*, la reproducibilidad exige cuidar la secuencia de ejecución, documentar decisiones y evitar depender de variables invisibles o celdas ejecutadas fuera de orden.

La función `print()` tiene aquí un papel más importante de lo que parece: no solo muestra resultados, sino que ayuda a comunicar qué se está calculando. Etiquetar salidas, redondear valores con `round()` y explicar la interpretación de cada medida forma parte del trabajo profesional de una lingüista computacional.

1.13. Del prototipo a la función: por qué reutilizar código importa

Un prototipo es una primera versión operativa de una idea. En esta unidad, el prototipo calcula medidas de un texto concreto. Cuando el mismo procedimiento se quiere aplicar a muchos textos, surge la necesidad de convertirlo en una función. Una función encapsula una serie de pasos, recibe una entrada y devuelve una salida.

Aunque las funciones se trabajen más adelante con mayor profundidad, conviene anticipar su sentido: reducen repetición, disminuyen errores y permiten comparar textos de forma sistemática. En términos profesionales, pasar de un conjunto de celdas sueltas a una función es pasar de una demostración a una herramienta reutilizable.

1.14. Interpretación crítica: el número no sustituye el juicio lingüístico

El resultado numérico de una métrica debe leerse como una pista, no como una sentencia. Si un texto obtiene un índice alto de dificultad, puede deberse a oraciones largas, palabras polisilábicas o decisiones de segmentación. Pero también puede tratarse de un texto especializado bien escrito para

un público experto. La interpretación depende del destinatario, del género discursivo y del propósito comunicativo.

Esta unidad, por tanto, no enseña solo Python. Enseña una forma de responsabilidad analítica: cada medida debe poder explicarse, cada regla debe poder justificarse y cada conclusión debe reconocer sus límites. Esa actitud será necesaria en todas las unidades posteriores de PLN.

2. Python para PLN: del texto como dato al primer prototipo de legibilidad

2.1. Google Colab: el laboratorio de trabajo

Comenzamos trabajando en el entorno virtual de Google Colab, que se compone de *notebooks*. Un *notebook* combina dos tipos de celdas:

- **Celdas de código**, donde escribimos instrucciones Python.
- **Celdas de texto**, donde explicamos qué estamos haciendo, formulamos hipótesis, documentamos resultados o redactamos conclusiones.

Esta combinación es especialmente valiosa en PLN porque el análisis de textos no es solo técnico. Hay que justificar decisiones. Por ejemplo, si decides que un punto y coma cuenta como frontera entre oraciones, esa decisión no pertenece solo a Python: pertenece también a tu interpretación lingüística de la puntuación.

El primer código del cuaderno es mínimo:

```
print("Bienvenido a tu primer cuaderno interactivo en Google Colab!")
```

La función `print()` muestra un mensaje en pantalla. Puede parecer un gesto pequeño, pero introduce una idea importante: un programa no solo calcula; también comunica resultados a una persona.

En un contexto profesional, un buen cuaderno no es el que «funciona en secreto», sino el que permite a otra persona del equipo entender qué hace cada celda, por qué se ejecuta en ese orden y cómo interpretar la salida.

2.2. El texto como dato: variables y strings

Vamos a guardar un primer texto en una variable:

```
texto1 = """El lenguaje claro es una herramienta fundamental para garantizar la
accesibilidad informativa;
todos los ciudadanos tienen derecho a comprender los mensajes institucionales sin
barreras lingüísticas."""
```

Una variable es un nombre que usamos para guardar un objeto. En este caso, el nombre es `texto1` y el objeto guardado es una cadena de texto, es decir, un `string`.

La elección del nombre importa. `texto1` indica que este fragmento será el primer texto de trabajo. En análisis lingüístico computacional, los nombres claros reducen errores. No es lo mismo trabajar con `x`, `y` y `z` que con `texto_original`, `texto_adaptado` y `texto_traducido`.

El texto aparece entre comillas triples:

```
"""texto largo"""
```

Las comillas triples permiten escribir cadenas de varias líneas. Son útiles para guardar párrafos, transcripciones, textos institucionales, fragmentos literarios o ejemplos de corpus.

Después, el cuaderno imprime el texto:

```
print("Texto original:")  
print(texto1)
```

Aquí hay dos operaciones distintas:

- `print("Texto original:")` muestra una etiqueta para que la salida sea comprensible.
- `print(texto1)` muestra el contenido almacenado en la variable.

Esta costumbre de etiquetar las salidas es básica en trabajos reproducibles. Cuando el cuaderno crece, no basta con que aparezcan números en pantalla. Hay que saber qué significa cada número.

2.3. De texto continuo a palabras: `split()`

El primer análisis que vamos a hacer consiste en contar `palabras`. Para hacerlo, Python necesita convertir el texto continuo en unidades separadas.

```
palabras = texto1.split()
```

El método `.split()` divide una cadena de texto en partes. Por defecto, separa donde encuentra espacios en blanco: espacios, saltos de línea o tabulaciones. El resultado es una lista.

Si `texto1` es un `string` completo, `palabras` pasa a ser una colección de unidades:

```
["El", "lenguaje", "claro", "es", "una", ...]
```

Desde el punto de vista del PLN, esto es una **tokenización básica**. Tokenizar significa dividir un texto en unidades de análisis. En esta unidad usamos una versión sencilla: separar por espacios.

Conviene ser precisa: `.split()` no entiende la lengua. Solo aplica una regla mecánica. Por eso puede dejar pegados signos de puntuación a las palabras:

```
"informativa;".split()
```

El resultado conservaría `informativa;` como una unidad con punto y coma incluido. Para muchas tareas iniciales basta, pero para análisis más finos necesitaremos tokenizadores más sofisticados.

La pregunta importante para una lingüista no es solo «cómo lo separa Python», sino «si esa separación representa bien el fenómeno que quiero estudiar».

2.4. Contar elementos con `len()`

Una vez que tenemos una lista de `palabras`, podemos contar sus elementos:

```
num_palabras = len(palabras)
print("\nNúmero de palabras:", num_palabras)
```

La función `len()` devuelve la longitud de un objeto. Si recibe una lista, cuenta cuántos elementos tiene. Si recibe un `string`, cuenta cuántos caracteres tiene. Si recibe un diccionario, cuenta cuántas entradas contiene.

En este caso:

```
len(palabras)
```

responde a la pregunta: cuántos elementos hay en la lista `palabras`.

Para el `texto1` del cuaderno, el resultado es:

Medida	Resultado
Número de <code>palabras</code>	25

Este número parece muy simple, pero ya convierte una impresión lectora en una medida observable. A partir de aquí se puede comparar: un texto tiene 25 `palabras`, otro tiene 19, otro tiene 80. La comparación ya no depende solo de la sensación subjetiva de longitud.

2.5. Importar herramientas: el módulo `string`

Python trae muchas funciones incorporadas, pero también permite importar módulos. Un módulo es un conjunto de herramientas ya preparadas.

Para contar `signos` de puntuación, el cuaderno importa `string`:

```
import string
```

El módulo `string` pertenece a la biblioteca estándar de Python. No hay que instalarlo. Dentro de él encontramos una constante muy útil:

```
string.punctuation
```

Esta constante contiene `signos` de puntuación comunes, como punto, coma, punto y coma, dos puntos, interrogación inglesa, paréntesis, comillas y otros símbolos.

La idea de fondo es profesional: no hace falta escribir siempre todo desde cero. Parte de programar consiste en saber qué herramientas existen, importarlas y usarlas de forma consciente.

Pero también hay que conocer sus límites. `string.punctuation` está pensado sobre todo para puntuación ASCII. En textos españoles puede que algunos `signos`, variantes tipográficas o caracteres especiales necesiten tratamiento adicional. Este detalle importa cuando trabajamos con corpus reales.

2.6. Recorrer caracteres y detectar puntuación

Con esta línea detectamos los `signos` de puntuación:

```
signos = [c for c in texto1 if c in string.punctuation]
```

Esta construcción se llama **lista por comprensión**. Es una forma compacta de crear una lista a partir de otra secuencia.

Podemos leerla así:

11. Recorre cada carácter `c` dentro de `texto1`.
12. Comprueba si ese carácter está en `string.punctuation`.
13. Si lo está, guárdalo en la lista `signos`.

La misma idea escrita en lenguaje natural sería:

```
"Dame todos los caracteres del texto que sean signos de puntuación."
```

En el `texto1`, la lista resultante es:

```
[';', ' ', '.']
```

Después se cuenta la lista:

```
num_signos = len(signos)
```

El resultado es:

Medida	Resultado
Signos encontrados	;, .
Número de <code>signos</code> de puntuación	2

Este bloque introduce tres ideas esenciales para seguir aprendiendo Python:

- Un `string` puede recorrerse carácter por carácter.
- `in` permite comprobar su pertenencia.
- Una lista por comprensión permite filtrar datos de manera concisa.

En PLN, estas tres ideas aparecen continuamente: buscar caracteres, detectar `palabras`, filtrar etiquetas, quedarse con determinados `tokens` o extraer patrones.

2.7. Contar oraciones: una decisión lingüística dentro del código

Ahora vamos a contar `oraciones` de forma muy simple:

```
oraciones = texto1.split(".")
num_oraciones = len([s for s in oraciones if s.strip()])
print("Número estimado de oraciones:", num_oraciones)
```

La primera línea divide el texto cada vez que encuentra un punto. Esto genera una lista de fragmentos.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

La segunda línea filtra esos fragmentos:

```
[s for s in oraciones if s.strip()]
```

El método `.strip()` elimina espacios al principio y al final de un `string`. Si un fragmento queda vacío, no se cuenta. Esto evita sumar falsas `oraciones` cuando el texto termina en punto o contiene espacios sobrantes.

Hasta aquí, Python aplica una regla formal: «divide por puntos». El problema es que la lengua real usa más `signos` para organizar el discurso. En el `texto1` aparece un punto y coma:

```
... accesibilidad informativa;  
todos los ciudadanos ...
```

Desde una lectura lingüística, ese punto y coma separa dos segmentos oracionales relevantes. Python no lo sabe. Por eso el cuaderno corrige el procedimiento:

```
oraciones = texto1.replace(";", ".").split(".")  
num_oraciones = len([s for s in oraciones if s.strip()])
```

Primero se reemplaza el punto y coma por un punto. Después se divide por puntos.

Esta celda es una de las más importantes de la unidad porque muestra que programar no es solo escribir instrucciones. Programar PLN es **traducir una decisión lingüística a una operación computacional**.

Si en otro texto quisiéramos tratar también los dos puntos como frontera, podríamos encadenar reemplazos:

```
oraciones = texto.replace(";", ".").replace(":", ".").split(".")
```

La regla mejora, pero no se vuelve perfecta. Abreviaturas como `Sr.`, decimales como `3.14` o puntos suspensivos pueden complicar el conteo. Por eso hablamos de número estimado de `oraciones`, no de número definitivo.

2.8. Expresiones regulares: buscar patrones de vocales

Para estimar sílabas, introducimos el módulo `re`:

```
import re
```

`re` permite trabajar con expresiones regulares. Una expresión regular es una forma de describir patrones de texto. No busca una palabra concreta, sino una forma.

El patrón usado en el cuaderno es:

```
r'[aeiouáéíóúü]+'
```

Y se aplica así:

```
silabas = re.findall(r'[aeiouáéíóú]+', texto1, re.IGNORECASE)
num_silabas = len(silabas)
```

Vamos por partes.

`re.findall()` busca todas las coincidencias de un patrón dentro de un texto y devuelve una lista.

El patrón contiene:

Fragmento	Significado
<code>r"</code>	Cadena <i>raw</i> : Python interpreta el patrón de forma literal
<code>[aeiouáéíóú]</code>	Cualquier vocal simple, acentuada o con diéresis
<code>+</code>	Uno o más caracteres del grupo anterior
<code>re.IGNORECASE</code>	Ignora mayúsculas y minúsculas

La intuición lingüística es esta: muchas sílabas contienen un núcleo vocálico. Si buscamos grupos de vocales, podemos aproximar el número de sílabas.

Por ejemplo, en *educación*, el patrón detecta grupos vocálicos. Ese recuento no equivale siempre a una silabificación correcta, pero sirve para construir una primera medida automatizada.

Esta guía debe insistir en la palabra aproximada. El español tiene diptongos, hiatos, sinalefas, casos dialectales y fenómenos fonéticos que este patrón no resuelve. Aun así, el ejercicio es muy valioso porque enseña cómo pasar de una observación lingüística, «las sílabas dependen de vocales», a una regla computacional operativa.

Para el `texto1`, obtenemos:

Medida	Resultado
Número aproximado de sílabas	69

2.9. Construir un índice de legibilidad aproximado

Una vez calculadas `palabras`, `oraciones` y `sílabas`, combinamos las medidas:

```
indice = (num_silabas / num_palabras) + (num_palabras / num_oraciones)
print("\nÍndice aproximado de legibilidad:", round(indice, 2))
```

La fórmula suma dos promedios:

Parte de la fórmula	Qué mide	Interpretación
<code>num_silabas / num_palabras</code>	Promedio de sílabas por palabra	Aproximación a dificultad léxica
<code>num_palabras / num_oraciones</code>	Promedio de palabras por oración	Aproximación a complejidad sintáctica

El primer término se relaciona con la longitud fonológica media de las `palabras`. Si las `palabras` tienen muchas sílabas, es posible que el texto sea más complejo, aunque no siempre.

El segundo término se relaciona con la longitud media de las oraciones. Las oraciones muy largas suelen exigir más memoria de trabajo y pueden dificultar la comprensión.

Para el `texto1`, los valores son:

Medida	Resultado
Palabras	25
Sílabas aproximadas	69
Oraciones estimadas	2
Índice aproximado	15.26

Este índice no es una fórmula oficial de legibilidad. Es un **prototipo didáctico**. Sirve para aprender a construir una métrica, no para certificar profesionalmente que un texto es accesible. Esta distinción es clave en segundo curso. Una métrica computacional puede orientar, comparar y detectar casos llamativos, pero no sustituye el juicio lingüístico. Un texto puede tener un índice alto y ser claro si usa vocabulario familiar. Otro puede tener frases cortas y ser difícil si contiene términos técnicos, ambigüedad o conceptos abstractos.

2.10. Redondear resultados con `round()`

A continuación, mostramos el índice así:

```
round(indice, 2)
```

`round()` redondea un número. El segundo argumento indica cuántos decimales queremos conservar. En informes para clientes o actividades docentes, mostrar 15.2633333333 no aporta claridad. Mostrar 15.26 es más legible.

Este detalle conecta con una competencia profesional: presentar resultados de forma comprensible. El análisis no termina cuando Python calcula. Termina cuando una persona puede leer el resultado y entender qué significa.

2.11. Reutilizar código: el paso previo a las funciones

Cuando un bloque se repite varias veces, aparece una pregunta profesional: por qué no convertirlo en una función. Todavía no desarrollaremos la función, pero una versión podría ser:

```
def analizar_legibilidad(texto):
    palabras = texto.split()
    num_palabras = len(palabras)
    signos = [c for c in texto if c in string.punctuation]
    oraciones = texto.replace(";", ".").replace(":", ".").split(".")
    num_oraciones = len([s for s in oraciones if s.strip()])
    silabas = re.findall(r'[aeiouáéíóú]+', texto, re.IGNORECASE)
    num_silabas = len(silabas)
    indice = (num_silabas / num_palabras) + (num_palabras / num_oraciones)
    return round(indice, 2)
```

Una función permite reutilizar el análisis sin copiar y pegar tantas líneas. En proyectos reales, esto reduce errores y facilita comparar muchos textos.

2.12. Cómo mejoraría LinguaAdapt este prototipo

Si el cliente quisiera convertir el cuaderno en una herramienta profesional, el equipo podría ampliarlo de varias formas:

- Usar un tokenizador lingüístico para separar palabras y signos con más precisión.
- Aplicar un segmentador de oraciones en lugar de reemplazos manuales.
- Incorporar un silabificador más fiable para español.
- Calcular longitud media de palabra y longitud media de oración por separado.
- Medir diversidad léxica con una relación tipo-*token*.
- Detectar vocabulario especializado o poco frecuente.
- Identificar estructuras sintácticas complejas, como subordinadas o pasivas.
- Crear gráficos sencillos para comparar textos.
- Exportar resultados a una tabla para analizar muchos documentos.

Estas mejoras muestran hacia dónde se dirige el curso: de operaciones básicas sobre `strings` a procedimientos más robustos de análisis lingüístico computacional.

Errores frecuentes de aprendizaje

Olvidar ejecutar celdas anteriores.

Si una variable como `texto1` no se ha definido, Python no puede usarla. En *notebooks*, el orden visual no siempre coincide con el orden real de ejecución.

Confundir el texto con el nombre de la variable.

`texto1` no es la frase. Es el nombre que apunta al objeto `string` donde está guardada la frase.

Pensar que `.split()` hace análisis lingüístico completo.

`.split()` separa por espacios. No interpreta morfología, puntuación ni sintaxis.

Contar `oraciones` sin filtrar fragmentos vacíos.

Después de dividir por puntos, pueden quedar cadenas vacías. Por eso se usa `if s.strip()`.

Interpretar el índice como diagnóstico absoluto.

El índice orienta, pero debe contrastarse con una lectura cualitativa.

Síntesis final

En esta unidad has aprendido la base conceptual de un primer análisis de PLN con Python: representar un texto como cadena de caracteres, guardarlo en una variable, dividirlo en unidades, contar rasgos observables y combinar esos rasgos en una medida aproximada.

La idea central es que programar no consiste solo en escribir instrucciones, sino en convertir decisiones lingüísticas en operaciones explícitas. Decidir qué cuenta como palabra, oración, signo de puntuación o grupo vocálico forma parte del análisis.

El prototipo de legibilidad no pretende sustituir una evaluación experta de accesibilidad. Sirve para comprender cómo se construye una métrica, por qué hay que documentar sus supuestos y cómo Python puede ayudar a comparar textos de forma inicial y reproducible.

Cierre narrativo

Al cerrar esta unidad, tu primer encargo en LinguaAdapt ya tiene forma. Has partido de un texto institucional, lo has guardado como dato, lo has dividido en unidades, has contado rasgos observables y has construido una métrica sencilla para orientar una decisión de accesibilidad.

Lo más importante no es el número final. Lo importante es haber entendido el proceso: cada resultado computacional nace de una decisión previa. Decidir qué cuenta como palabra, qué cuenta como oración o qué patrón aproxima una sílaba son elecciones que conectan conocimiento lingüístico y programación. Ese es el punto de entrada al PLN: usar Python para hacer explícitas nuestras hipótesis sobre el lenguaje, probarlas sobre textos reales y revisar críticamente lo que la máquina devuelve.

Python no lee como una lingüista. Pero una lingüista que sabe Python puede enseñarle a observar patrones, medirlos y convertirlos en preguntas mejores.

Referencias

- Biber, D., Conrad, S. y Reppen, R. (1998). *Corpus linguistics: Investigating language structure and use*. Cambridge University Press.
- Bird, S., Klein, E. y Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media. <https://www.nltk.org/book/>
- Fernández Huerta, J. (1959). Medidas sencillas de lecturabilidad. *Consigna*, 214, 29-32.
- Flesch, R. (1948). A new readability yardstick. *Journal of Applied Psychology*, 32(3), 221-233. <https://doi.org/10.1037/h0057532>
- Google. (2026). Colaboratory: Frequently asked questions. <https://research.google.com/colaboratory/faq.html>
- Gries, S. T. (2009). *Quantitative corpus linguistics with R: A practical introduction*. Routledge.
- Jupyter Notebook Team. (2026). Jupyter Notebook documentation. <https://jupyter-notebook.readthedocs.io/>
- Jurafsky, D. y Martin, J. H. (2026). *Speech and language processing* (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C. y Jupyter Development Team. (2016). Jupyter Notebooks: A publishing format for reproducible computational workflows. En F. Loizides y B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>
- Python Software Foundation. (2026a). Built-in functions. Python 3.14 documentation. <https://docs.python.org/3/library/functions.html>
- Python Software Foundation. (2026b). Built-in types: Text sequence type str. Python 3.14 documentation. <https://docs.python.org/3/library/stdtypes.html>
- Python Software Foundation. (2026c). string: Common string operations. Python 3.14 documentation. <https://docs.python.org/3/library/string.html>
- Python Software Foundation. (2026d). re: Regular expression operations. Python 3.14 documentation. <https://docs.python.org/3/library/re.html>
- Szigriszt Pazos, F. (1993). *Sistemas predictivos de legibilidad del mensaje escrito: Fórmula de perspicuidad* [Tesis doctoral, Universidad Complutense de Madrid]. <https://produccioncientifica.ucm.es/documentos/5d1df5fb29995204f765de7f>